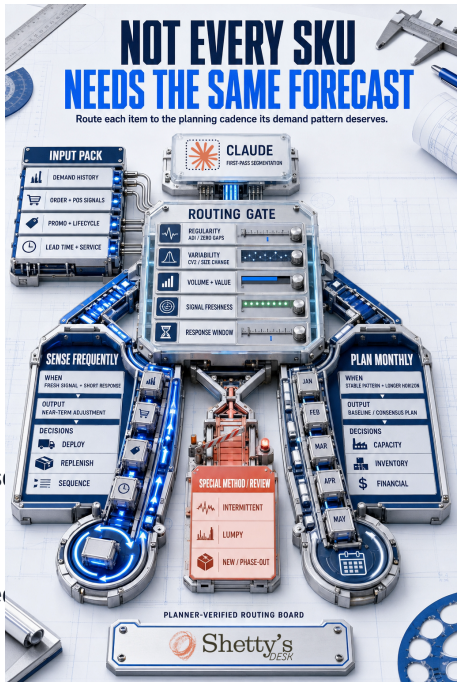
FREE WORKFLOW PACK - VERSION 1.1.0

Demand Sensing Router

A beginner-safe guide to run the sample, replace it with your own data, and prepare a planner-reviewable routing board.



Start with synthetic data and two standard-library Python s

Choose a chat-first, Agent Skill, or Claude Project path.

Route each SKU-location with traceable evidence and che

Keep final policy and system changes with the planner.

WHO THIS IS FOR

Demand planners, supply-chain analysts, planning leaders, and transformation teams trying the workflow for the first time.

10 MIN SAMPLE	2 INPUT FILES	5 REVIEW OUTPUTS

1 - CHOOSE A ROUTE

Pick the simplest first-run path

Run the synthetic sample before deciding how deeply to install the workflow. The operating method stays the same; only the tool surface changes.

Path	Use it when	What to do first
A - Chat first	You want to understand the method with the least setup.	Run the profile script, upload four files, and paste the supplied chat prompt.
B - Agent Skill	Your tool supports a skills folder or Agent Skills.	Install the complete skill folder, then run the sample and validator.
C - Project / Cowork	A business user needs one reusable controlled workspace.	Attach the skill, workers, and inputs; use the Cowork run prompt manually.

RECOMMENDED FIRST RUN Choose Path A. It proves the data shape and routing logic before you install or schedule anything.

What this workflow builds

A current routing board for each SKU-location.

A route-change log showing what moved and why.

A data-quality exception list for missing or stale evidence.

A demand-review brief connecting changes to open decisions.

A run manifest with source dates, policy version, and validation status.

2 - RUN THE SAMPLE

Prove the package before using your data

Open Terminal, move into the extracted package folder, and run:

```
python3 skills/demand-sensing-router/scripts/profile_demand.py
skills/demand-sensing-router/assets/sample-demand-history.csv
--output demand-profile.csv
```

Expected result:

```
Wrote 4 demand profiles to demand-profile.csv
```

Then validate the supplied routing-board example:

```
python3 skills/demand-sensing-router/scripts/validate_routing.py
examples/routing-board-example.csv
```

Expected result:

```
Routing validation passed
```

What success means

Python can read the sample data and create one profile per SKU-location.

The validator can read the expected routing-board schema.

You have not connected production data or changed any planning policy.

3 - REPLACE FILE ONE

Map your demand history

Copy the sample instead of overwriting it:

```
sample-demand-history.csv -> my-demand-history.csv
```

Column	Replace with	Rule
sku	Your item, material, or product code	Use one stable identifier.
location	Planning location, plant, DC, or market	Use the planning grain used in review.
period	Daily or weekly period	Use one consistent ISO date or week format.
demand	Historical actual demand	Use numeric demand for that period.

Keep one row per SKU-location-period.

Do not mix daily and weekly history for the same SKU-location.

Use zero for genuine zero demand; do not use zero for missing data.

Anonymize identifiers before using an external AI service unless your approved enterprise environment permits the data.

4 - REPLACE FILE TWO

Map the routing context

Copy the sample instead of overwriting it:

```
sample-routing-context.csv -> my-routing-context.csv
```

Column	Replace with
sku, location	The same keys used in demand history
signal_type	POS, orders, promotion, launch plan, or another approved signal
signal_date	The date the signal was observed or extracted
signal_freshness_days	Days between the signal date and review date
response_window_days	Days remaining for the supported decision to change
lifecycle	Active, phase-in, phase-out, or your governed equivalent

Column	Replace with
volume_value_tier	Your approved portfolio tier
service_priority	High, medium, low, or your approved service class
decision_supported	Replenishment, deployment, allocation, capacity, sequence, or method choice
planner_comment	A short known event, assumption, or one-off explanation
previous_cadence	The previously approved routing lane

NON-NEGOTIABLE Use ISO dates such as 2026-07-15. A signal without a date cannot pass the freshness check.

5 - RUN YOUR DATA

Create the first routing package

Profile your history:

```
python3 skills/demand-sensing-router/scripts/profile_demand.py
my-demand-history.csv
--output my-demand-profile.csv
```

Give the assistant these four files:

my-demand-profile.csv

my-routing-context.csv

skills/demand-sensing-router/references/routing-policy.md

skills/demand-sensing-router/references/data-contract.md

Then paste this instruction:

```
Use the supplied routing policy and data contract. Combine my demand profile with my routing context.
Produce the routing board, route-change log, data-quality exceptions, demand-review brief, and run
manifest. Do not invent missing evidence. Do not publish a frequent-sensing route without a dated
signal, an open response window, and a supported operating decision. Send failed checks to planner
review.
```

For a tool-specific setup, use the complete prompt in prompts/chat-first-starter.md or prompts/cowork-run-prompt.md.

6 - REVIEW THE OUTPUTS

Start the meeting with exceptions

Output	Use it for	Review first
Routing board	The current lane and its evidence	Changed and failed rows
Route-change log	What moved since last cycle	Every unexplained move
Data-quality exceptions	Missing, stale, or conflicting evidence	All open exceptions
Demand-review brief	Decision-focused meeting preparation	Closing response windows
Run manifest	Traceability and reproducibility	Source dates and validation status

Planner approval questions

What evidence caused this SKU-location to move?

Is the trigger signal still fresh enough to matter?

Can replenishment, deployment, allocation, capacity, or sequence still change?

What evidence is missing or contradictory?

Should the proposed lane be approved, rejected, or held for special review?

DECISION BOUNDARY The workflow prepares evidence and a recommendation. The planner approves forecast policy, cadence, master data, and every system-of-record change.

7 - FIX COMMON ISSUES

Troubleshooting and final check

Problem	What to check
python3: command not found	Install Python 3 or use a managed company environment.
Missing-column error	Compare your header row with the sample exactly.
Duplicate-row error	Keep one row per SKU-location-period.
Unsupported frequent-sensing route	Add a dated signal, open response window, and decision it can still change.
Mixed daily and weekly history	Separate the series or convert it to one consistent bucket.
Sensitive data concern	Anonymize identifiers or use an approved enterprise AI environment.

Final checklist

- [ ] The sample scripts run successfully.
- [ ] My two input files preserve the supplied column names.
- [ ] Every signal has a source date.
- [ ] Every frequent-sensing route names an open decision window.
- [ ] Changed routes explain why they moved.
- [ ] Failed checks remain exceptions.
- [ ] A planner approves the final cadence and system update.

NEXT

Run the sample, inspect the five example outputs, and only then make copies of the two CSV input files. Keep the original examples as your known-good reference.